EE5203 L06 Study Sheet

GPIO Interrupts - Basri Erdogan

Essential question: How does one B1 press travel from a voltage change on PC13 all the way into your C code — without the main loop ever asking about the button?

What you should be able to do

- Walk the five-stage path with real names: PRESS → NOTICE (EXTI) → ASK (IRQ/NVIC) → LOOK UP (vector table) → RUN YOUR CODE (handler → callback).
- Configure a falling-edge EXTI pin in CubeMX and read the generated code.
- Explain `IMR`, `FTSR`, `PR`, and write-1-to-clear.
- Write a short callback that records an event in a `volatile` flag.
- Consume the event in `main()` and debounce with `HAL_GetTick()`.

The path, with real names

```
B1 press  ->  PC13 falls          (active-low: released=1, pressed=0)
          ->  EXTI13 edge detect  (SYSCFG->EXTICR[3] field = 0010 routes PC13)
          ->  pending bit set     (EXTI->PR bit 13 = 1)
          ->  switch 1: EXTI mask (EXTI->IMR bit 13 = 1)
          ->  EXTI15_10_IRQn      (one request shared by lines 10-15)
          ->  switch 2: NVIC      (enable + priority, delivers to CPU)
          ->  CPU accepts         (finishes instruction, saves its place)
          ->  vector table        (IRQ number -> handler address)
          ->  EXTI15_10_IRQHandler()        (processor enters this)
          ->  HAL_GPIO_EXTI_IRQHandler()    (clears PR, then...)
          ->  HAL_GPIO_EXTI_Callback(pin)   (your code - HAL calls this)
```

EXTI is an ST peripheral; NVIC, CPU and the vector table belong to the Arm Cortex-M4 core. The request reaches the CPU only when both enable switches are closed (bit = 1). Only one port can drive a given EXTI line at a time — the SYSCFG routing field stores exactly one choice (`0000` PA13, `0001` PB13, `0010` PC13).

Vocabulary table

Name	Job
EXTI	hardware edge detector; remembers the event in the pending bit
IRQ (`EXTI15_10_IRQn`)	the numbered request "this needs attention"
NVIC	enables, prioritizes, and delivers requests to the CPU
Vector table	maps each interrupt number to its handler's address
Handler (`..._IRQHandler`)	the function the **processor** enters
Callback (`..._Callback`)	the function **HAL** calls; you write it

Configuration (CubeMX → generated code)

Three choices: PC13 = `GPIO_EXTI13`; falling edge + **no pull** (external R30); NVIC checkbox `EXTI line[15:10] interrupts`.

```c
GPIO_InitStruct.Mode = GPIO_MODE_IT_FALLING;
GPIO_InitStruct.Pull = GPIO_NOPULL;
HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);   /* writes SYSCFG, IMR, FTSR */

HAL_NVIC_SetPriority(EXTI15_10_IRQn, 0, 0);
HAL_NVIC_EnableIRQ(EXTI15_10_IRQn);
```

Register, bit 13 = 1	Meaning	Who writes it
`EXTI->IMR`	line may raise an interrupt	`HAL_GPIO_Init`
`EXTI->FTSR`	falling edge selected	`HAL_GPIO_Init`
`EXTI->PR`	edge pending, unserviced	hardware sets; software clears by **writing 1**

Breakpoint inside the handler → `PR` bit 13 still reads `1` (the HAL helper has not cleared it yet).

The event-flag pattern

```c
volatile uint8_t button_event = 0;          /* ISR-shared -> volatile   */
uint32_t last_press_ms = 0;

void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    uint32_t now = HAL_GetTick();
    if (GPIO_Pin == GPIO_PIN_13 && (now - last_press_ms) >= 80) {
        last_press_ms = now;
        button_event = 1;                   /* record - nothing else    */
    }
}

/* in while (1): */
if (button_event == 1) {
    button_event = 0;                       /* consume BEFORE acting    */
    HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);  /* application work here    */
}
```

- **Callback rules:** check the source, record, return fast. No `HAL_Delay`, no loops, no application logic.
- **`volatile`:** the interrupt changes the variable between two main-loop instructions; without it the compiler may keep using a stale copy.
- **Debounce:** contacts bounce for ~10 ms → several falling edges. 80 ms ignores all of them yet allows fast intentional presses. The unsigned subtraction `now - last_press_ms` stays correct across tick wrap-around.
- **Two different "flags":** `EXTI->PR` bit 13 lives in hardware, cleared by the HAL helper (write-1-to-clear); `button_event` lives in your C program, cleared by `main()`.

SFR evidence

Claim	Evidence
PC13 routed to EXTI13	`SYSCFG->EXTICR[3]` line-13 field = `0010`
falling edge selected	`EXTI->FTSR` bit 13 = `1`
line unmasked	`EXTI->IMR` bit 13 = `1`
edge awaiting service	`EXTI->PR` bit 13 = `1` while paused in the handler

Common mistakes

- Rising edge selected; internal pull fighting the board circuit.
- NVIC checkbox forgotten — EXTI sets `PR`, nothing is delivered.
- Callback compares the wrong pin mask.
- Long callback (delays/logic) blocking everything else.
- Missing `volatile` on the shared flag.
- No debounce — one press, several events.

Mastery self-check

☐ I can recite the path from press to callback with real names.
☐ I can state the three CubeMX choices and the registers they write.
☐ I can explain write-1-to-clear and where the pending bit lives.
☐ I can say which function the processor enters and which HAL calls.
☐ I can justify `volatile` in one sentence about stale copies.
☐ I can trace a bouncy press through the 80 ms guard.
☐ I can predict `EXTI->PR` bit 13 at a handler breakpoint.

Five review questions

1. The NVIC checkbox is left unchecked. Which registers still change on a press, and what never happens?
2. Why does the *release* of B1 not toggle the LED in the lab program?

3. A student calls `HAL_Delay(200)` inside the callback "for debouncing." Give two distinct reasons this is wrong.
4. Edges arrive at t = 5000, 5003, 5008, 5012, 5240, 5310 ms. With the 80 ms guard, which are accepted, and what is `last_press_ms` at the end?
5. `button_event` works in a debug build but the LED stops responding at `-O2`, and adding `volatile` fixes it. Explain what the optimizer did.

See the L06 worksheet for extended practice. Worked solutions are released after the practice window.